

Лабораторна робота №5

Мета роботи

Нейтралізація надлишковості даних та усунення аномалій модифікації шляхом покрокового проведення нормалізації проєктованої бази даних пункту прокату фільмів. Визначення функціональних залежностей, аналіз початкової схеми на відповідність критеріям першої (1NF), другої (2NF) та третьої (3NF) нормальних форм, виконання декомпозиції проблемних сутностей та розробка фінального DDL-скрипту.

1. Початкова ненормалізована схема даних (Денормалізований вигляд)

Для демонстрації повного життєвого циклу процесу нормалізації та виконання критеріїв оцінювання, розглянемо початковий архітектурний варіант, у якому вся інформація системи зберігалася у двох масивних денормалізованих таблицях: INITIAL_FILM (каталог стрічок) та INITIAL_RENTAL (журнал замовлень та транзакцій).

Структура таблиці INITIAL_FILM:

- film_id (Первинний ключ) — унікальний код фільму.
- title — назва фільму.
- release_year — рік випуску.
- rental_rate — вартість оренди.
- categories — список жанрів, записаних через кому (наприклад: 'Sci-Fi, Action, Drama').

Структура таблиці INITIAL_RENTAL:

- rental_id (Первинний ключ) — номер операції прокату.
- rental_date — дата видачі.
- return_date — дата повернення.
- customer_id — ідентифікатор клієнта.
- customer_name — повне ім'я клієнта.
- customer_email — електронна адреса клієнта.
- inventory_id — унікальний штрихкод конкретного диска.
- staff_id — ідентифікатор співробітника.
- staff_name — ім'я касира.
- staff_position — посада співробітника.
- staff_status — статус зайнятості співробітника.
- payment_amount — сума оплати.
- payment_status — статус чека.

2. Функціональні залежності (ФЗ) для початкової схеми

Для проведення коректного аналізу визначимо мінімальний повний набір функціональних залежностей (ФЗ) неключових атрибутів від ключових для обох проблемних сутностей.

ФЗ для таблиці INITIAL_FILM:

1. film_id >> title, release_year, rental_rate
2. film_id >> categories (проте атрибут є неатомарним)

ФЗ для таблиці INITIAL_RENTAL:

1. rental_id >> rental_date, return_date, customer_id, inventory_id, staff_id, payment_amount, payment_status
2. customer_id >> customer_name, customer_email
3. staff_id >> staff_name, staff_position, staff_status
4. inventory_id >> film_id

3. Визначення поточної нормальної форми та аномалій

Таблиця INITIAL_FILM перебуває в недонормалізованому стані (нижче 1NF).

- **Порушення:** Стовець categories містить масиви або списки значень, розділених комою. Це прямо порушує правило атомарності атрибутів першої нормальної форми.
- **Аномалія вибірки:** Пошук усіх фільмів жанру 'Action' вимагає використання повільних операторів тексту (LIKE '%Action%'), що критично знижує швидкість OLTP-систем.

Таблиця INITIAL_RENTAL перебуває в першій нормальній формі (1NF).

- **Порушення 2NF:** Таблиця не має часткових залежностей від свого первинного ключа rental_id, оскільки він є простим (не складеним). Відповідно, формально вона задовольняє 2NF, але містить критичні транзитивні залежності.
- **Порушення 3NF (наявність транзитивних залежностей):**
 - rental_id >> customer_id >> customer_email (Пошта залежить від клієнта, а не від факту оренди).
 - rental_id >> staff_id >> staff_position (Посада залежить від касира, а не від оренди).
- **Аномалії модифікації даних:**
 - *Аномалія вставки (Insertion):* Ми не можемо зареєструвати в системі нового співробітника чи клієнта, поки вони не здійснять хоча б одну реальну операцію оренди диска (оскільки rental_id є первинним ключем і не може бути NULL).
 - *Аномалія видалення (Deletion):* Якщо клієнт поверне свій єдиний диск і ми видалимо запис про оренду, з бази безслідно зникнуть персональні дані цього клієнта (customer_name, customer_email).
 - *Аномалія оновлення (Update):* Якщо касир змінить посаду, нам доведеться оновлювати рядок staff_position у тисячах старих архівних записів оренди, що призведе до ризику неузгодженості даних.

4. Покроковий процес нормалізації

Крок 1: Перехід до 1NF (Забезпечення атомарності)

Щоб усунути множинні значення в таблиці INITIAL_FILM, декомпонуємо списки. Кожен жанр записується в окремий рядок. Первинний ключ стає складеним (компонентним).

- **Проміжний дизайн таблиці FILM_1NF:**

FILM_1NF (*film_id*, *category_name*, title, release_year, rental_rate)

Тепер дані атомарні. Таблиця в 1NF.

Крок 2: Перехід до 2NF (Усунення часткових залежностей)

У таблиці FILM_1NF є складений ключ (*film_id*, *category_name*). Атрибути title, release_year та rental_rate залежать лише від *частини* цього ключа — від *film_id*. Це часткова залежність, яку необхідно усунути шляхом розбиття.

- **Результат декомпозиції для 2NF:**

1. FILM (*film_id*, title, release_year, rental_rate) — ключ *film_id*.
2. CATEGORY (*category_id*, name) — виділяємо довідник жанрів.
3. FILM_CATEGORY (*film_id*, *category_id*) — зв'язуюча таблиця для реалізації відношення «багато до багатьох».

Крок 3: Перехід до 3НФ (Усунення транзитивних залежностей)

Тепер реорганізуємо таблицю INITIAL_RENTAL. Виносимо всі транзитивні залежності у самостійні батьківські таблиці, де кожен неключовий атрибут залежатиме *виключно* від свого первинного ключа.

1. Виносимо дані клієнта: CUSTOMER (*customer_id*, first_name, last_name, email).
2. Виносимо дані персоналу: STAFF (*staff_id*, first_name, last_name, position, status).
3. Виносимо сутність фізичної копії: INVENTORY (*inventory_id*, *film_id*, condition).
4. Виносимо фінансові транзакції: PAYMENT (*payment_id*, amount, payment_date, status, rental_id, customer_id).

В основній таблиці оренди залишаються лише прямі зовнішні ключі-посилання.

- **Остаточний набір таблиць у 3NF:**

- CATEGORY, FILM, FILM_CATEGORY, INVENTORY, CUSTOMER, STAFF, RENTAL, PAYMENT.
- Будь-які аномалії усунено: тепер кожен факт (профайл клієнта, ціна фільму чи статус працівника) зберігається в базі даних **рівно один раз**.

5. Модифікація структури та фінальний SQL DDL-скрипт (3NF)

Для приведення бази даних до третьої нормальної форми (3NF) на фізичному рівні СКБД було виконано повне очищення та перестворення таблиць. Нижче наведено фінальний скрипт, що завантажується в pgAdmin без помилок.

```
CREATE TYPE disk_condition AS ENUM ('New', 'Good', 'Damaged', 'Lost');
```

```
CREATE TYPE staff_position AS ENUM ('Cashier', 'Manager', 'Administrator');
```

```
CREATE TYPE staff_status AS ENUM ('Active', 'On_Leave', 'Temporary');
```

```
CREATE TYPE payment_status AS ENUM ('Completed', 'Delayed', 'Cancelled');
```

```
CREATE TABLE CATEGORY (  
    category_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE FILM (  
    film_id SERIAL PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    release_year INT CHECK (release_year >= 1895 AND release_year <= 2100),  
    rental_rate DECIMAL(5, 2) NOT NULL  
);
```

```
CREATE TABLE FILM_CATEGORY (  
    film_id INT REFERENCES FILM(film_id) ON DELETE CASCADE,  
    category_id INT REFERENCES CATEGORY(category_id) ON DELETE CASCADE,  
    PRIMARY KEY (film_id, category_id)  
);
```

```
CREATE TABLE INVENTORY (  
    inventory_id SERIAL PRIMARY KEY,  
    film_id INT REFERENCES FILM(film_id) ON DELETE RESTRICT,  
    condition disk_condition NOT NULL DEFAULT 'New'  
);
```

```
CREATE TABLE CUSTOMER (  
    customer_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE CHECK (email LIKE '%@%')  
);
```

```
CREATE TABLE STAFF (  
    staff_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    position staff_position NOT NULL,  
    status staff_status NOT NULL DEFAULT 'Active'  
);
```

```
CREATE TABLE RENTAL (  
    rental_id SERIAL PRIMARY KEY,  
    rental_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    return_date TIMESTAMP,  
    customer_id INT REFERENCES CUSTOMER(customer_id) ON DELETE RESTRICT,  
    inventory_id INT REFERENCES INVENTORY(inventory_id) ON DELETE RESTRICT,  
    staff_id INT REFERENCES STAFF(staff_id) ON DELETE RESTRICT  
);
```

```
CREATE TABLE PAYMENT (  
    payment_id SERIAL PRIMARY KEY,  
    amount DECIMAL(7, 2) NOT NULL CHECK (amount > 0),  
    payment_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    status payment_status NOT NULL DEFAULT 'Completed',  
    rental_id INT REFERENCES RENTAL(rental_id) ON DELETE CASCADE,  
    customer_id INT REFERENCES CUSTOMER(customer_id) ON DELETE RESTRICT  
);
```

6. Оновлена діаграма реляційної схеми

Нова нормалізована структура повністю відображає архітектуру третьої нормальної форми (3NF), де відсутні дублювання текстових полів та транзитивні зв'язки.

